GALGOTIAS UNIVERSITY
(Established under Galgotias University Uttar Pradesh Act No. 14 of 2011)

NAAC GRADE A+ Accredited University

Plot No. 2, Sector 17-A Yamuna Expressway, Greater Noida, Gautam Buddha Nagar, Uttar Pradesh, India.

SECTION 47

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE NAME

OBJECT ORIENTED PROGRAMMING USING

JAVA

COURSE CODE (R1UC201C)

B.TECH. SECOND SEM (2024 -2025)

PROJECT TITLE: ONSLAUGHT (VISAGE VENGEANCE)

SUBMITTED BY

AMAN PATEL (24SCSE1260003)

KAUSTUBH KUMAR (24SCSE1330024)

SMITH SHUKLA (24SCSE1330010)

SUBITTED TO

PROF.

MR. TARACHAND VERMA

(OOPS USING JAVA)

INTRODUCTION

ONSLAUGHT

“VISAGE VENGEANCE”

"Onslaught: Visage Vengeance" is a **Java-based 2D action shooter game** that combines futuristic gameplay with real-time face recognition for secure login and personalized experience.

Set in a dystopian world where identity is power, players take on the role of a resistance fighter battling through hostile environments to uncover a deep conspiracy. The game features smooth player controls, dynamic enemy waves, and an engaging progression system.

🔧 **Key Features:**

- 🎮 **Built with JavaFX** for user interface elements like menus, login, and profile selection.
- 🔫 **Game Engine using LibGDX** for high-performance 2D graphics and game logic.
- 🧠 **Face Recognition Login** using OpenCV and deep learning (JavaCV) for secure and immersive access.
- 🧱 Modular code architecture for easy expansion and maintenance.
- 🎨 Pixel-art style environment with layered parallax effects and responsive sound design.

PRIMARY USES OF THE ONSLAUGHT (VISAGE VENGEANCE)

🎮 GAME DEVELOPMENT WITH JAVA:
This project showcases the use of Java for creating a dynamic 2D action game. With JavaFX for UI and LibGDX for game mechanics, it demonstrates Java's capabilities in real-time game development.

🧠 FACE RECOGNITION INTEGRATION:
The game integrates face recognition for login using JavaCV and OpenCV, offering secure authentication and a personalized experience for each player.

📚 EDUCATIONAL RESOURCE FOR GAME DEVELOPMENT:
This project serves as an educational tool for learning game development, real-time mechanics, and integrating biometric authentication. It helps developers understand the application of game physics, collision detection, and user interaction.

🔐 BIOMETRIC SECURITY IN GAMING:
Face recognition as a login method demonstrates how biometric security can be incorporated into games, adding layers of personalization and security.

💻 SHOWCASING JAVA'S VERSATILITY:
The project highlights Java's ability to handle both UI design and complex game mechanics, proving its use in performance-critical applications like games.

🕵 ENHANCING USER ENGAGEMENT:
By using face recognition, the game adapts to each player's identity, enhancing user experience and fostering deeper engagement with the game world.

🚀 EXPANDING JAVA'S APPLICATION SCOPE:
"Onslaught" demonstrates how Java can be applied in gaming, breaking out of traditional desktop applications into the realm of interactive, real-time games.

⚔ INNOVATIVE GAMEPLAY MECHANICS:
The unique face recognition feature introduces a new level of interactivity, making the player's identity a central part of the gaming experience.

PROJECT REQUIREMENTS FOR

"ONSLAUGHT: (VISAGE VENGEANCE)"

1. HARDWARE REQUIREMENTS:

- **Processor:** Intel Core i3 or higher (or equivalent AMD processor)
- **RAM:** Minimum 4 GB (8 GB recommended)
- **Storage:** 1 GB of free disk space for game installation and asset
- **Graphics:** Dedicated graphics card with at least 1 GB VRAM (e.g., NVIDIA GeForce or AMD Radeon)
- **Camera:** For face recognition, a **webcam** with at least 720p resolution is required

2. SOFTWARE REQUIREMENTS:

- **Operating System:** Windows 10/11, macOS, or Linux
- **Java Development Kit (JDK):** Version 11 or higher
- **Integrated Development Environment (IDE):** Eclipse, IntelliJ IDEA, or NetBeans
- **Libraries and Frameworks:**
  - **LibGDX:** For game mechanics and graphics rendering
  - **JavaFX:** For the user interface and controls
  - **JavaCV/OpenCV:** For face recognition integration
- **Database:** SQLite or similar (optional for storing user profiles and game data)

3. DEVELOPMENT TOOLS:

- **Version Control:** Git and GitHub for version tracking and collaboration
- **Build Tools:** Apache Maven or Gradle for project dependencies and builds
- **Image Editing Software:** For assets like sprites and textures (e.g., Adobe Photoshop, GIMP)
- **Audio Editing Software:** For sound effects and music (e.g., Audacity, FL Studio)

4. NETWORK REQUIREMENTS:

- **Internet Connection:** Required for downloading dependencies, libraries, and updates

5. ADDITIONAL REQUIREMENTS:

- **Camera Calibration:** A system for configuring and testing face recognition before gameplay
- **Testing and Debugging Tools:** Unit testing frameworks (e.g., JUnit) for Java, visual testing tools for the UI
- **Game Asset Management System:** To organize and manage game assets (images, sounds, and animations) during development
- **Performance Optimization Tools:** Profiling tools (e.g., VisualVM) to monitor the performance of the game and optimize for smooth gameplay.

6. SECURITY AND PRIVACY:

- **Data Privacy Management:** Ensure user data (especially facial data) is securely handled and complies with data protection regulations (e.g., GDPR).
- **Encryption:** For face recognition data storage and transmission to maintain privacy and security.

7. USER DOCUMENTATION:

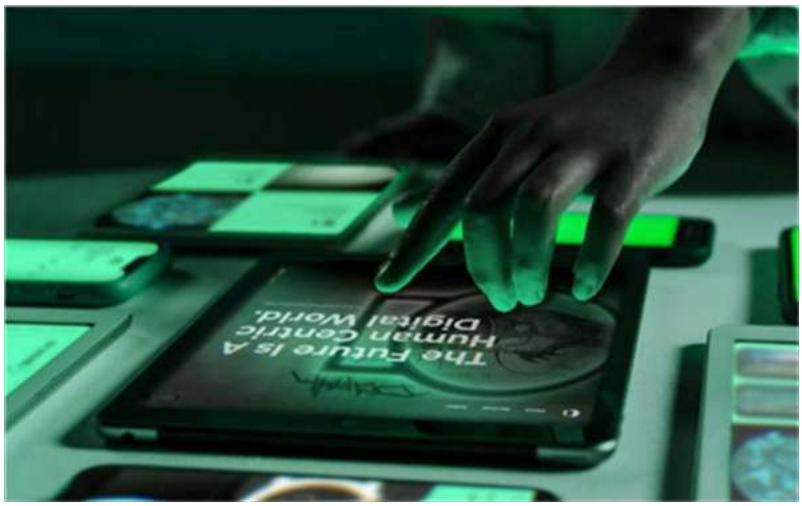
- **Help and Tutorial System:** An in-game guide or manual explaining how to use the face recognition login system and controls.
- **User Feedback Mechanism:** A system for collecting user feedback to improve gameplay and usability.

COMPONENTS OF THE "ONSLAUGHT: VISAGE VENGEANCE"

USER INTERFACE (UI):

The **UI** component is responsible for providing an interactive experience for the player. Using **JavaFX**, this component handles elements like the **main menu**, **settings screen**, **profile selection**, and **face recognition login**. The UI ensures smooth navigation and user-friendly interactions throughout the game.

- **Login Screen**: Integration of **face recognition** for user authentication
- **Menu System**: Main menu, pause menu, and settings options
- **HUD (Heads-Up Display)**: Displays vital game stats such as health, ammo, and score.



GAME ENGINE (LIBGDX):

The **game engine** is responsible for the core game mechanics. Built using **LibGDX**, this component handles the game's **physics engine**, **collision detection**, **input management**, and **sprite rendering**. It allows for real-time gameplay with smooth animations and interactions.

- **Game Loop**: The continuous cycle that updates the game state and renders the game's visuals
- **Player Controls**: Movement, shooting, and interacting with the environment
- **Enemy AI**: Behavior patterns for enemies that create challenges for the player

FACE RECOGNITION SYSTEM:

This is a key feature of the game, utilizing **JavaCV** and **OpenCV** libraries to implement **face recognition** technology. The system authenticates the player's identity during the login process, enhancing security and providing a personalized experience.

- **Face Detection**: Capturing the player's face using the webcam
- **Face Matching**: Comparing the captured face with stored data for authentication
- **Profile Linking**: Linking the recognized face with the player's saved game profile

GAME ASSETS:

Game assets include all the visual and auditory elements required for the game. These assets will be managed and optimized for performance using various editing tools and asset management systems.

- **Visual Assets**: Sprites, backgrounds, animations, and textures
- **Audio Assets**: Background music, sound effects for actions like shooting and explosions
- **Lighting and Effects**: Special effects like particle systems, lighting, and shadows to enhance visual appeal.

DATABASE/STORAGE SYSTEM:

The game needs to store player data, including profiles, progress, and settings. A simple **database** (like **SQLite**) or a **file-based storage system** will be used to persist these details

- **Player Profiles**: Storing player preferences, stats, and progress
- **Game Settings**: Saving and loading sound, difficulty, and display settings
- **High Scores and Achievements**: Tracking player milestones an rankings

MULTIPLAYER (OPTIONAL):

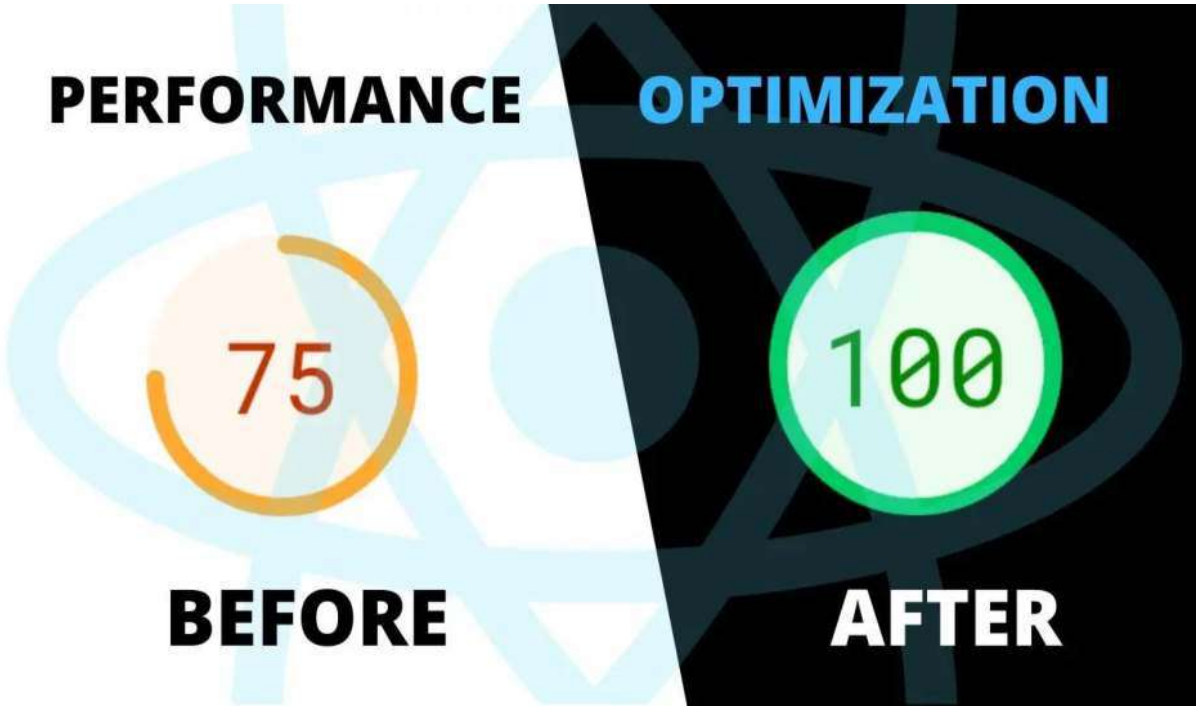
If multiplayer features are added, this component will handle **network communication** between players. It will manage the connection, synchronization of player states, and interaction within the game.

- **Server Management**: Handling multiple connections and data exchange
- **Matchmaking System**: Connecting players to multiplayer sessions

PERFORMANCE OPTIMIZATION:

This component is responsible for ensuring the game runs smoothly across various systems. **Profiling tools** (such as **VisualVM**) and **optimization techniques** will be used to ensure minimal latency and resource consumption.

- **Memory Management**: Efficient handling of assets and in-gam objects to avoid performance drops
- **Frame Rate Optimization**: Ensuring consistent frame rates for smooth gameplay
- **CPU/GPU Utilization**: Monitoring and optimizing the game's use of system resources



SECURITY AND PRIVACY:

As face recognition data is being used, the game needs a **secure system** for data storage and transmission to protect user privacy. Encryption and secure handling of personal data will be a priority.

- **Data Encryption**: Encrypting face data and personal informatio to protect user privacy
- **Secure Data Handling**: Ensuring compliance with data protection regulations like GDPR

CONCLUSION

"Onslaught: Visage Vengeance" is more than just a game—it is a **technological demonstration** of how traditional programming languages like **Java** can evolve to support **modern interactive applications**. The successful integration of **face recognition** not only enhances the **security and personalization** of the game but also opens doors for future applications in other industries like **edtech**, **healthcare**, and **authentication systems**.

This project has helped showcase:

- The **versatility of Java** in combining user interfaces, real-time processing, and advanced features.
- How **game development** can be a powerful educational tool for learning programming, AI, and UI design.
- The future potential of **biometric technology** in enhancing user experience and data protection.

By bridging **gaming and facial recognition**, this project sets an example of innovation in digital interaction, user identity, and immersive design.

OBJECTIVE

The primary objective of **"Onslaught: Visage Vengeance"** is to design and develop a **2D action shooter game** using Java that integrates **face recognition technology** for secure and personalized user login. This project aims to combine **biometric authentication** with interactive game mechanics to provide a **futuristic and immersive gaming experience**.

Key goals include:

- Demonstrating **Java's capability** in handling real-time game development using frameworks like **LibGDX** and **JavaFX**.
- Implementing **face detection and recognition** using **JavaCV/OpenCV** for secure, user-specific login.
- Creating engaging game elements such as **player movement**, **enemy AI**, and **HUD systems**.
- Offering students and developers a practical understanding of **game architecture**, **UI/UX design**, and **security integration**.
- Highlighting the potential of **Java in modern application areas** beyond traditional software, including entertainment and biometric tech.